

8-2 · DAG 이해 및 유즈케이스 연구 · 이론 1H

# DAG 핵심 개념

레시피 순서도로 잡는 DAG·Task·Operator

수도권ICT 이노베이션스퀘어  
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

# 1시간 뒤에는

---

- ① DAG·Task·Operator 의 관계를 그림으로 그려 설명한다
- ② >> 와 리스트로 순서·병렬·합류 의존성을 쓴다
- ③ `schedule·start_date·catchup` 의 뜻을 알고 `catchup=False` 를 쓴다
- ④ 멍등성이 재시도의 전제 조건인 이유를 말한다

다음 1.5H 실습에서 DAG 세 개를 직접 만들며 순서·병렬·재시도·실패를 겪어 봅니다

1. 세 낱말

# DAG · Task · Operator — 레시피 한 장으로

| 낱말       | 무엇인가       | 비유        |
|----------|------------|-----------|
| DAG      | 순서도 전체     | 레시피 한 장   |
| Task     | 순서도의 상자 하나 | 「양파를 볶는다」 |
| Operator | 상자를 만드는 틀  | 조리 도구     |

PythonOperator 는 파이썬 함수를, BashOperator 는 셸 명령을 태스크로 만든다

DAG 는 「무엇을 어떤 순서로」를 적은 파이썬 파일 — Task 는 실행 단위이자 재시도 단위

## 1. 세 낱말

# 설계의 핵심 — Task 를 얼마나 잘게 나눌까

1

### 너무 굵으면

실패했을 때 재시도 비용이 크다 — 처음부터 다시

2

### 너무 잘면

태스크가 많아져 관리가 번거롭다

3

### 기준 하나

「여기까지는 성공한 채로 다시 시작하고 싶은가?」

임베딩이 비싸다면

자르기와 임베딩을  
나눈다

자르기를 다시 하지 않아도 되니까 — 비싼 단계 앞에서 자른다

2. 의존성

# >> 로 순서를, 리스트로 병렬을

| 코드                   | 뜻               |
|----------------------|-----------------|
| t1 >> t2             | t1 이 성공해야 t2 시작 |
| t1 >> [t2, t3]       | t1 뒤에 둘이 병렬로    |
| [t2, t3] >> t4       | 둘 다 끝나야 t4      |
| t1 >> [t2, t3] >> t4 | 갈라졌다 다시 합류      |

화살표 방향만 있고 되돌아가는 길은 없다(Acyclic) — t1 >> t2 >> t1 은 순환이라 Airflow 가 거부한다

3. 스케줄

# schedule 값 다섯 가지 — 학습에는 None

| schedule 값   | 뜻                   |
|--------------|---------------------|
| None         | 수동 트리거 전용 (학습에 편하다) |
| "@daily"     | 매일 자정 1회            |
| "@hourly"    | 매시 정각               |
| "0 6 * * *"  | 크론 표기 — 매일 06:00    |
| "30 6 * * 1" | 매주 월요일 06:30        |

start\_date 는 이 DAG 가 언제부터 유효한가 — 과거 날짜를 넣으면 소급 실행 대상이 된다

### 3. 스케줄

# catchup 과 논리적 날짜 — 처음에 반드시 헷갈리는 둘

## catchup — 밀린 날짜 소급

start\_date 부터 오늘까지 밀린 날짜를 전부 실행할지

- True 면 과거 실행이 수백 개 쏟아진다
- 학습 중에는 항상 catchup=False
- 실습 DAG 마다 명시한다

VS

## 논리적 날짜 — 한 주기 앞

@daily 로 1월 2일 자정에 도는 실행의 날짜는 1월 1일

- 「1월 1일치 데이터를 처리하는 작업」이라서
- 사람이 생각하는 날짜와 다르다
- 지금 기억해 두면 나중에 안 해맨다

4. 멍등성

# 같은 태스크를 같은 날짜로 다시 돌려도 결과가 같아야

| 주차  | 멍등을 어떻게 만들었나                       |
|-----|------------------------------------|
| 4주차 | MERGE — 있으면 갱신, 없으면 삽입             |
| 7-4 | uuid5(경로+번호)로 id 고정 → upsert 가 덮어씀 |
| 8주차 | 위 둘을 태스크 안에 넣어 재시도가 안전해진다          |

재시도가 기본인 도구 — 「재시도해도 되는가」를 태스크마다 확인한다

실패한 태스크만 골라 재실행하는 Airflow 운영 방식의 전제 조건 — 멍등하지 않으면 데이터가 두 번 들어간다

## 5. 재시도

# default\_args 로 재시도 두 줄 — 언제 쓰고 언제 안 통하나

재시도의 전제

태스크가  
역등할 것

- 1 retries: 2**  
실패하면 2번 더 시도한다
- 2 retry\_delay: timedelta(minutes=1)**  
1분 간격으로 — DAG(..., default\_args=default\_args)
- 3 네트워크·API 같은 일시적 실패에 유용**  
코드 오류는 몇 번을 재시도해도 같으니 로그를 봐야 한다

# 이 1시간에서 꼭 가져갈 것

---

- 1 DAG 는 순서도, Task 는 상자, Operator 는 상자를 만드는 틀
- 2 나누는 기준 — 여기까지는 성공한 채로 다시 시작하고 싶은가
- 3 >> 로 순서를, 리스트로 병렬을 — 순환은 안 된다
- 4 학습 DAG 는 catchup=False 를 반드시 명시한다
- 5 논리적 날짜는 사람이 생각하는 날짜보다 한 주기 앞선다
- 6 재시도가 기본인 도구이므로 태스크는 멍등해야 한다

8-2 · 이론 1H 마무리

## 다음은 첫 DAG 작성 실습 (1.5H)

hello\_dag·etl\_practice·fan\_practice 세 DAG 를 만들며 순서·병렬·재시도·실패를 직접 겪어 봅니다

✓ dags/ 에 저장하고 웹 UI 목록에서 확인

✓ airflow tasks test 로 단위 실행

✓ 중간 태스크를 일부러 실패시키고 Clear

✓ 순환을 만들어 거부되는 것 보기

준비할 것

~/airflow-lab Airflow 기동 상태